

Cloud IT Online — Written Exam

Assignment: Answers

Q1. Compare BFS & DFS with examples.

Breadth-First Search (BFS) explores a graph level by level, visiting all neighbors of a node before moving to the next depth. It uses a **queue (FIFO)**. **Depth-First Search (DFS)** explores as far as possible along a branch before backtracking. It uses a **stack** (or recursion).

Aspect	BFS	DFS
Data structure	Queue	Stack / Recursion
Memory	$O(b^d)$ — can be high (stores whole frontier)	$O(d)$ — lower, stores only path
Shortest path (unweighted)	Guaranteed	Not guaranteed
Use case	Shortest path, level-order traversal	Topological sort, cycle detection, maze solving

Example: Graph A-B, A-C, B-D, C-D. - BFS from A: $A \rightarrow B, C \rightarrow D$ (visits by level) - DFS from A: $A \rightarrow B \rightarrow D \rightarrow C$ (goes deep first, backtracks)

Q2. Time complexity of Merge Sort (Best, Average, Worst)

Merge Sort always splits the array in half and merges — its behavior doesn't depend on input order, so: - **Best case:** $O(n \log n)$ - **Average case:** $O(n \log n)$ - **Worst case:** $O(n \log n)$

Space complexity: $O(n)$ (auxiliary array for merging). It is a stable, comparison-based, divide-and-conquer algorithm — recurrence: $T(n) = 2T(n/2) + O(n)$.

Q3. Preprocessing a dataset with missing values and noise

- Handle missing values:**
 - Remove rows/columns with excessive missing data.
 - Impute using mean/median/mode (numeric/categorical), or model-based imputation (KNN, regression).
 - Use a “missing” indicator flag if missingness itself is informative.
 - Handle noise:**
 - Apply smoothing (binning, moving average, regression smoothing).
 - Detect and treat outliers (Z-score, IQR method, clustering-based outlier detection).
 - Other steps:** normalization/standardization, removing duplicates, encoding categorical variables, and feature scaling before feeding into a model.
-

Q4. B+ Tree, order $m = 4$, height 0 to height 2 — max keys

For a B+ tree of order $m = 4$, each node can have at most $m - 1 = 3$ keys and at most $m = 4$ child pointers.

- **Height 0 (root only, leaf):** max 3 keys.
- **Height 1 (root + 1 level of children):** root has up to 4 children, each holding up to 3 keys $\rightarrow 4 \times 3 = 12$ keys (leaf level), plus root's own keys don't duplicate data in a B+ tree (root keys are just separators). Counting **all nodes across the tree structure** (root + level 1 + level 2):
 - Level 0 (root): 3 keys
 - Level 1: up to 4 nodes $\times$ 3 keys = 12 keys
 - Level 2: up to $4 \times 4 = 16$ nodes $\times$ 3 keys = 48 keys

Maximum total keys stored across all nodes (height 0 to 2) = $3 + 12 + 48 = 63$.

(Note: in a pure B+ tree, only leaf-level keys hold actual data records; internal keys are duplicated separators. If the question intends *only distinct data records* stored at the leaf level (height 2), the answer is **48**.)

Q5. 2PL vs Strict 2PL for T1, T2

T1: R(X), W(X), R(Y), W(Y) T2: R(Y), W(Y), R(X), W(X)

Standard 2PL (Two-Phase Locking): - Each transaction has a *growing phase* (acquiring locks) and a *shrinking phase* (releasing locks); once a lock is released, no new locks may be acquired. - Locks can be released as soon as they are no longer needed — even before commit. - This guarantees **conflict serializability**, but not necessarily that reads/writes are isolated until commit — it can suffer from **cascading rollbacks** (if T1 releases a lock on X, T2 reads it, and then T1 aborts, T2 must also be rolled back). - With T1 and T2 above, if T1 releases its lock on X right after W(X) (before committing), T2 can immediately R(X)/W(X), creating a dependency that risks cascading aborts.

Strict 2PL: - All locks (read and write) are held until the transaction **commits or aborts** — no early release. - This prevents cascading rollbacks and provides **recoverability**, at the cost of reduced concurrency (T2 must wait until T1 fully commits before touching X or Y if there's a conflict). - In the example, T2 could not acquire the lock on Y until T1 releases it at commit, forcing more serialization/waiting than plain 2PL.

Summary: Both guarantee serializability, but Strict 2PL trades concurrency for safety (no cascading aborts), which is why it's the protocol most real DBMS use.

Q6. TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable — acknowledgments, retransmission,	Unreliable — no guarantee of delivery

	ordering	or order
Speed	Slower (overhead of handshake, ACKs, flow/congestion control)	Faster (minimal overhead)
Use case	Web browsing (HTTP/HTTPS), email, file transfer	Real-life example: Video/voice calls (VoIP) and live video streaming, where speed matters more than occasional lost packets

Q7. Fingerprint system for a banking app — one functional & one security requirement

- **Functional requirement:** The system shall allow a registered user to log in by scanning their fingerprint via the device's fingerprint sensor; matching it against the stored biometric template, and granting access to the account within 2 seconds on a successful match.
 - **Security requirement:** The system shall never store the raw fingerprint image; only an encrypted, irreversible biometric template/hash shall be stored (e.g., in the device's secure enclave), and fingerprint data shall never leave the device or be transmitted to the bank's servers.
-

Q8. Inheritance in OOP

Inheritance allows a class (subclass/derived class) to acquire the properties and behaviors (fields and methods) of another class (superclass/base class), promoting code reuse and establishing an "is-a" relationship.

Three common types: 1. **Single Inheritance:** One subclass inherits from one superclass. (e.g., class Dog extends Animal) 2. **Multilevel Inheritance:** A class inherits from a class which itself inherits from another. (e.g., Animal → Mammal → Dog) 3. **Multiple Inheritance:** A class inherits from more than one superclass. (Not directly supported in Java for classes, but achieved via interfaces; supported directly in C++.)

(Other types sometimes discussed: Hierarchical inheritance — multiple subclasses from one superclass; Hybrid — combination of the above.)

Q9. IP addressing, routing, and OSPF at the network layer

Each device on a network is identified by an **IP address**, which routers use to determine the destination network for a packet.

Routing is the process of selecting a path across interconnected networks to deliver packets from source to destination.

OSPF (Open Shortest Path First) is a link-state interior gateway routing protocol that works as follows: 1. Each router discovers its neighbors and builds a **Link-State Advertisement (LSA)** describing its directly connected links and their costs. 2. Routers flood these LSAs to all other routers in the area, so every router builds an

identical **Link-State Database (LSDB)** representing the full network topology. 3. Each router independently runs **Dijkstra's Shortest Path First algorithm** on this database to compute the shortest-cost path to every other network/router. 4. The result populates the router's **routing table**, mapping destination IP prefixes to the best next-hop interface. 5. When a packet arrives, the router looks up the destination IP in its routing table (longest prefix match) and forwards it to the next hop determined by OSPF's shortest path — this repeats hop-by-hop until the packet reaches its destination.

OSPF converges quickly and scales well because it uses areas to limit LSA flooding scope.

Q10. Universal Gate & Proof that NAND is Universal

A **Universal Gate** is a logic gate that can be used to implement any Boolean function / any other basic gate (AND, OR, NOT) using only combinations of itself. NAND and NOR are the two universal gates.

Proof NAND is universal (derive NOT, AND, OR using only NAND):

1. **NOT using NAND:** Tie both inputs of a NAND gate together. NOT $A = \text{NAND}(A, A) = \neg(A \cdot A) = \neg A$
2. **AND using NAND:** A NAND gate followed by a NOT (itself a NAND): $\text{NAND}(A, B) = \neg(A \cdot B)$ AND $(A, B) = \text{NOT}(\text{NAND}(A, B)) = \text{NAND}(\text{NAND}(A, B), \text{NAND}(A, B)) = A \cdot B$
3. **OR using NAND:** By De Morgan's law, $A + B = \neg(\neg A \cdot \neg B)$. So invert A and B first (using NAND-as-NOT), then NAND them:
 $\text{OR}(A, B) = \text{NAND}(\text{NOT}(A), \text{NOT}(B)) = \text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B)) = A + B$

Since NOT, AND, and OR (a functionally complete set) can all be built from NAND alone, **NAND is a universal gate**.

Q11. Version Control (Git) — Commit vs Push

Version Control is a system that records changes to files over time, allowing developers to track history, revert changes, and collaborate without overwriting each other's work. **Git** is the most widely used distributed version control system.

- **Committing** saves a snapshot of your staged changes to your **local repository** only. It creates a permanent checkpoint in your local history with a message describing the change. No one else sees it yet.
- **Pushing** uploads your local commits to a **remote repository** (e.g., GitHub/GitLab), making them available to collaborators and syncing the remote branch with your local branch.

In short: commit = save locally; push = share remotely.

Q12. Binary Search — step by step, and why it needs a sorted array

Steps: 1. Set low = 0, high = n - 1. 2. While low <= high: a. Compute mid = (low + high) / 2. b. If array[mid] == target, return mid (found). c. If array[mid] < target, search the right half: low = mid + 1. d. If array[mid] > target, search the left half: high = mid - 1. 3. If the loop ends without a match, the target is not present.

Each step halves the search space, giving **$O(\log n)$** time complexity.

Why it fails on an unsorted array: Binary search relies on the ordering property — deciding to discard the left or right half assumes everything on one side is smaller and the other side larger than the midpoint. If the array isn't sorted, that assumption breaks, and the target could be anywhere, including in the half being discarded — so the algorithm may skip over the correct answer entirely.

Q13. Implementing a Queue (FIFO) using two Stacks

Use two stacks, `inStack` and `outStack`.

- **Enqueue(x):** Simply push `x` onto `inStack`. $O(1)$.
- **Dequeue():**
 - If `outStack` is empty, pop every element from `inStack` and push it onto `outStack` (this reverses the order, so the oldest element ends up on top of `outStack`).
 - Pop and return the top of `outStack`.

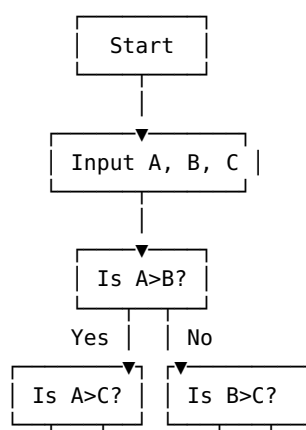
Why it works: `inStack` holds elements in the order they were inserted (newest on top). When transferred to `outStack`, the order is reversed, placing the oldest element on top — matching FIFO order. Elements are only transferred when `outStack` is empty, so each element moves between stacks at most twice — giving **amortized $O(1)$** per operation.

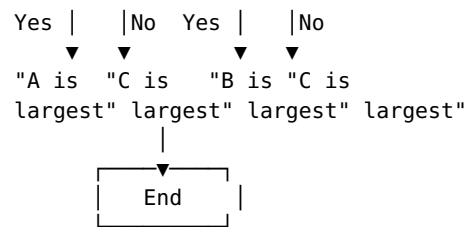
Q14. Bubble Sort logic & inefficiency vs Merge Sort

Logic: Repeatedly step through the array, comparing each pair of adjacent elements and swapping them if they are in the wrong order. After each full pass, the largest unsorted element “bubbles” to its correct position at the end. Repeat for $n-1$ passes (or until no swaps occur, indicating the array is sorted).

Why it's inefficient for large datasets: - Time complexity is **$O(n^2)$** in the average and worst case (nested loops comparing/swapping), compared to Merge Sort's **$O(n \log n)$** . - Bubble Sort does many redundant comparisons and swaps, one element at a time, while Merge Sort divides the problem in half recursively and merges efficiently. - For large n , $O(n^2)$ grows dramatically faster than $O(n \log n)$ — e.g., for $n = 1,000,000$, Bubble Sort does $\sim 10^{12}$ operations vs Merge Sort's $\sim 2 \times 10^7$ — making Bubble Sort impractical at scale.

Q15. Flowchart — largest of three distinct numbers (A, B, C)





Logic: Compare A and B first. Whichever is bigger, compare it against C; the winner of that comparison is the overall largest.

Q16 (heuristic search). Why A* can be faster than blind search (BFS) though not always optimal-complete-guaranteed

Argument: 1. BFS (a *blind*/uninformed search) expands nodes purely based on distance from the start (number of edges), with no knowledge of which direction leads toward the goal. It must explore **all** nodes at depth d before moving to depth $d+1$, so its search space grows exponentially with branching factor b and depth d : $O(b^d)$. 2. **A*** is an *informed* search that uses a heuristic function $h(n)$ estimating the cost from node n to the goal, combined with the actual cost so far $g(n)$, to prioritize expanding nodes with the lowest $f(n) = g(n) + h(n)$. 3. Because A* actively directs its search toward nodes that *seem* closer to the goal, it avoids expanding large portions of the search space that a blind search would explore uniformly — it “focuses” the search. 4. If the heuristic $h(n)$ is **admissible** (never overestimates true cost) and consistent, A* is guaranteed to find the optimal path while still exploring far fewer nodes than BFS. 5. However, if a heuristic is **inadmissible** (can overestimate), A* may find a *suboptimal* path faster — trading guaranteed optimality for speed, since it aggressively trusts the heuristic’s estimate and may commit to a promising-looking but ultimately longer path.

Conclusion: The efficiency gain comes from using extra domain knowledge (the heuristic) to prune the search space intelligently, rather than exploring blindly in all directions — this is why A* can vastly outperform BFS in practice, even though a poorly designed (inadmissible) heuristic sacrifices the guarantee of finding the shortest path.

Q17 (Library DB). Book-Borrower relationship

The relationship between **Book** and **Borrower** is **Many-to-Many (M:N)**.

Justification: A single borrower can borrow multiple different books over time (and potentially hold several simultaneously, depending on library policy), and a single book (or, more precisely, a book title with multiple physical copies) can be borrowed by many different borrowers over its lifetime. Since both sides can relate to many of the other, this is M:N.

In a relational schema, this M:N relationship should be implemented using a **junction/bridge table**, e.g., `Borrowing(borrow_id, book_id, borrower_id, borrow_date, return_date)`, where `book_id` and `borrower_id` are foreign keys referencing the Book and Borrower tables respectively.

Q18. Checksum logic and data integrity

A **checksum** is a small, fixed-size value computed from a block of data using a mathematical function (e.g., summing bytes, or a CRC/hash algorithm), used to detect errors introduced during transmission or storage.

How it verifies integrity during file transfer: 1. Before sending, the sender computes a checksum over the data (e.g., sum of all bytes, or a hash like MD5/SHA). 2. The sender transmits both the data and the checksum value. 3. Upon receiving, the receiver independently recomputes the checksum over the received data using the same algorithm. 4. The receiver compares its computed checksum with the one received. 5. If they **match**, the data is assumed intact (no error introduced). If they **don't match**, the data was corrupted in transit, and the receiver requests retransmission.

Checksums can't correct errors or guarantee against all possible corruption (some error patterns may cancel out), but they provide a cheap, effective first line of defense for detecting accidental transmission errors.

Q19. Supervised vs Unsupervised vs Reinforcement Learning

Paradigm	Data	Goal	Example
Supervised Learning	Labeled data (input-output pairs)	Learn a mapping function to predict labels for new inputs	Spam detection, house price prediction
Unsupervised Learning	Unlabeled data	Discover hidden structure/patterns (clusters, associations)	Customer segmentation, anomaly detection
Reinforcement Learning	No fixed dataset; an agent interacts with an environment	Learn a policy that maximizes cumulative reward through trial and error	Game-playing AI (e.g., AlphaGo), robotics control

Contrast: Supervised learning needs a “teacher” providing correct answers; unsupervised learning finds structure with no answers given at all; reinforcement learning learns from delayed feedback (rewards/penalties) through interaction, rather than static labeled examples.

Q20. Rectangle & square with equal area

- Square perimeter = 32 m $\rightarrow$ side = $32/4 = 8$ m.
- Square area = $8 \times 8 = 64$ m².
- Rectangle width = 4 m, and rectangle area = square area = 64 m².
- Rectangle length = Area / Width = $64 / 4 = 16$ meters.

Answer: 16 meters.

Q21. Real-life software project (sample answer — adapt with your own experience)

This is a personal/experience-based question — here is a sample structure you can adapt:

"I worked on a [web-based inventory management / e-commerce / attendance] system for [a client/university project]. Key problems faced: 1. **Scalability issue:** The database slowed down as data grew — solved by adding indexing and query optimization. 2. **Concurrent access conflicts:** Multiple users editing the same record caused inconsistent data — solved by implementing optimistic locking / transactions. 3. **Requirement changes mid-project:** Solved by adopting an Agile/iterative approach with regular client feedback."

(Replace with your actual project details for the exam — interviewers/examiners expect a real, specific example.)

Q22. Employees table — SQL queries

Table: Employees(employee_id, employee_name, department_id, salary)

a. Total number of employees in each department:

```
SELECT department_id, COUNT(*) AS total_employees
FROM Employees
GROUP BY department_id;
```

b. Average salary across all employees in the organization:

```
SELECT AVG(salary) AS average_salary
FROM Employees;
```

Q23. Function Point calculation

Given (average complexity weights): User Input = 4, User Output = 5, User Inquiries = 4, User Files = 10, External Interface = 7 (standard IFPUG average weights).

Component	Count	Avg Weight	Subtotal
User Input	50	4	200
User Output	40	5	200
User Inquiries	35	4	140
User Files	6	10	60
External Interface	4	7	28

Unadjusted Function Points (UFP) = 200+200+140+60+28 = 628

Since the **Complexity Adjustment Factor (CAF)** is "average," CAF = 1 (i.e., Value Adjustment Factor VAF = $0.65 + 0.01 \times \sum Fi$, and with all 14 factors at average value 3, $\sum Fi = 42$, so VAF = $0.65 + 0.42 = 1.07$ — but many textbooks simplify "average CAF" to **1.0** for this style of problem).

Function Point (FP) = UFP × CAF = 628 × 1 = 628 Function Points (If using VAF = 1.07: $FP \approx 628 \times 1.07 \approx 672$ function points — state your assumption clearly in the exam.)

Q24. CRC — data 11100, divisor 1001

CRC (Cyclic Redundancy Check) appends $r = (\text{divisor length} - 1)$ zero bits to the data, then performs modulo-2 (XOR) division by the divisor; the remainder is the CRC code.

Divisor 1001 has 4 bits → append 3 zero bits to data 11100 → 11100000.

Modulo-2 division:

11100000 ÷ 1001

```
11100000
1001
----
0110 0
 000 0
----
0110 00
 0000
----
01100 0 -> bring down systematically:
```

Let's do it step by step (XOR at each aligned position):

```
11100000 (dividend)
1001
----
0110      (11100 XOR 1001 = 0110... ) take first 4 bits 1110 XOR
1001 = 0111
```

Redo carefully, bit by bit:

Dividend: 1 1 1 0 0 0 0

Step 1: take first 4 bits = 1110. Since leading bit 1 → XOR with 1001:
1110 XOR 1001 = 0111

Bring down next bit (0): remainder becomes 1110 (0111·2 + next bit 0
→ "1110") Leading bit 1 → XOR with 1001: 1110 XOR 1001 = 0111

Bring down next bit (0): → 1110 Leading bit 1 → XOR with 1001 =
0111

Bring down next bit (0): → 1110 Leading bit 1 → XOR with 1001 =
0111

No more bits to bring down. Remainder = last 3 bits of 0111 = **111**

CRC remainder = 111

**Transmitted data = original data + CRC = 11100 111 =
11100111**

*(At the receiver, dividing the received codeword by the same divisor
1001 should give a remainder of 0 if no error occurred.)*

Q25. Smallest element in an array (code)

```
#include <stdio.h>

int findSmallest(int arr[], int n) {
    int smallest = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < smallest) {
            smallest = arr[i];
        }
    }
    return smallest;
}

int main() {
    int arr[] = {12, 4, 7, 1, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
        printf("Smallest element: %d\n", findSmallest(arr, n));  
        return 0;  
    }
```

Time complexity: $O(n)$, Space: $O(1)$.

Q26. Attacker steals the TLS private key — what can be done?

If an attacker obtains a website's TLS private key undetected, they can:

1. **Impersonate the server:** Set up a fake server presenting the stolen certificate/key, tricking clients into connecting to it (man-in-the-middle / phishing) — since the private key proves server identity in TLS.
2. **Decrypt past/future traffic:** If the TLS session used a key-exchange method without **Perfect Forward Secrecy (PFS)** (e.g., static RSA key exchange), the attacker can decrypt any previously captured encrypted traffic to/from that server. If PFS (e.g., ephemeral Diffie-Hellman) was used, past sessions remain safe, but the attacker can still intercept/decrypt *future* sessions until the key is revoked.
3. **Perform active MITM attacks:** Intercept and decrypt live traffic, modify data in transit, or inject malicious content while appearing to be the legitimate server.
4. **Undermine trust:** Issue seemingly-valid responses for that domain indefinitely until the certificate is revoked (via CRL/OCSP) and reissued.

Mitigation: Immediately revoke the compromised certificate, issue a new key pair/certificate, and use protocols with forward secrecy so past sessions are not retroactively compromised.

Q27. Frame A navigating Frame B (cross-origin iframe security policy)

Modern browsers restrict whether one frame (A) can navigate another frame (B) to a new origin. The reasoning behind allowing navigation based on **display/containment** (whether A's visible area actually contains B, and A has genuine control/ownership over that region) rather than purely on origin match is:

- It prevents **"framebusting"/clickjacking-style attacks** where an invisible or unrelated frame silently redirects a frame the user is actually looking at and interacting with, without the user's awareness.
- If frame A genuinely and visibly contains frame B on screen (i.e., B is a legitimate child region within A's visible layout) and A has structural control over B (e.g., A created B, or B is nested within A's DOM), it is reasonable to trust that A's intent to navigate B is a deliberate, expected part of the page's own UI/UX design — not a hidden malicious redirect.
- This policy defends against attacks where a malicious top-level page embeds a legitimate site in a hidden/tiny iframe and forces it to navigate somewhere else without visible evidence to the user, while still allowing normal legitimate use-cases (like a parent page controlling its own visibly-nested iframe, e.g., a tabbed UI or a wizard flow).

In short: origin alone isn't sufficient because same-origin doesn't imply "part of the same visible interaction"; **visibility and structural containment** better capture genuine user-facing intent versus deceptive redirection.

Q28. Adjacency List vs Adjacency Matrix — which is better for which problem

- **Adjacency List is more efficient for:**
 - **Sparse graphs** (few edges relative to vertices) — space is $O(V+E)$ instead of $O(V^2)$.
 - Traversal algorithms (BFS/DFS) — iterating over a vertex's neighbors is $O(\text{degree}(v))$ instead of $O(V)$.
 - Finding all neighbors of a vertex.
- **Adjacency Matrix is more efficient for:**
 - **Dense graphs** (many edges close to V^2) — simpler and comparably space-efficient.
 - **Checking if a specific edge (u, v) exists** — $O(1)$ lookup, vs $O(\text{degree}(u))$ in an adjacency list.
 - Algorithms needing frequent edge-existence checks, e.g., Floyd-Warshall all-pairs shortest path, which naturally operates on a matrix.

Q29. Hash table (size 13), $h(k) = k \bmod 13$, linear probing

Insert order: 10, 3, 6, 16, 17, 19, 10, 3, 6, 16, 17, 19

Initial hash values: $h(10)=10$, $h(3)=3$, $h(6)=6$, $h(16)=3$, $h(17)=4$, $h(19)=6$

Step	Key	$h(k)$	Action	Slot filled
1	10	10	slot 10 empty → insert	10
2	3	3	slot 3 empty → insert	3
3	6	6	slot 6 empty → insert	6
4	16	3	slot 3 taken → probe 4 → empty → insert	4
5	17	4	slot 4 taken → probe 5 → empty → insert	5
6	19	6	slot 6 taken → probe 7 → empty → insert	7
7	10 (dup)	10	slot 10 taken → probe 11 → empty → insert	11
8	3 (dup)	3	taken→4 taken→5 taken→6 taken→7 taken→probe 8 → empty → insert	8
9	6 (dup)	6	taken→7 taken→8 taken(just filled)→probe 9 → empty →	9

10	16 (dup)	3	insert 3,4,5,6,7,8,9 all taken → probe 12 → 12 empty → insert 4,5,6,7,8,9,12? not contiguous— continue probing from 4: 4,5,6,7,8,9 taken → 10 taken → 11 taken → 12 taken → wrap to 0 → empty → insert 6,7,8,9 taken →10,11,12 taken→0 taken(just filled)→1 → empty → insert	0
11	17 (dup)	4		
12	19 (dup)	6		1

Final table (index: key):

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Key	17	19	—	3	16	17	6	19	3	6	10	10	16

(Note: this hash table as described allows duplicate keys since it's a multiset/log of insertions rather than a set; each duplicate probes forward until it finds an empty slot, exactly as shown above.)

Q30. Man-in-the-middle attack on Diffie-Hellman (two key pairs)

In standard Diffie-Hellman, Alice and Bob exchange public values $g^a \bmod p$ and $g^b \bmod p$ to compute a shared secret $g^{ab} \bmod p$. Without authentication, this is vulnerable to MITM:

1. The attacker, Mallory, intercepts the communication channel between Alice and Bob and generates **two separate DH key pairs**: one to use with Alice ($x, g^x \bmod p$), and one to use with Bob ($y, g^y \bmod p$).
2. When Alice sends her public value $g^a \bmod p$ to Bob, Mallory intercepts it and instead sends **her own** public value $g^x \bmod p$ to Bob, pretending to be Alice.
3. Similarly, when Bob sends $g^b \bmod p$ toward Alice, Mallory intercepts it and sends her **other** public value $g^y \bmod p$ to Alice, pretending to be Bob.
4. Now Alice computes a shared secret $K1 = (g^y)^a \bmod p$ with Mallory (believing it's shared with Bob), and Bob computes a shared secret $K2 = (g^x)^b \bmod p$ with Mallory (believing it's shared with Alice).
5. Mallory, knowing both her private exponents x and y , can independently compute **both** $K1$ (using Alice's public value and her own y) and $K2$ (using Bob's public value and her own x).
6. Mallory now sits transparently in the middle: she decrypts messages from Alice (encrypted with $K1$), reads/modifies them, then re-encrypts with $K2$ and forwards to Bob, and vice versa —

while Alice and Bob believe they are communicating securely and directly with each other.

Root cause: plain DH provides no authentication of the public values exchanged, so it cannot detect that the values were substituted. This is why real protocols (e.g., TLS) combine DH with digital signatures/certificates to authenticate the exchanged public keys.

Q31. CIA (Confidentiality, Integrity, Availability) vs falsification, masquerade, denial of service

The CIA triad describes the properties we want to protect; the four listed attack types are the corresponding threats that violate them:

- **Confidentiality** — protects data from unauthorized disclosure. It is violated when information is exposed to those not authorized to see it (e.g., eavesdropping, or a **masquerade** where an attacker impersonates a legitimate user to gain unauthorized access to confidential data).
- **Integrity** — protects data from unauthorized modification. It is violated by **falsification**, where data is altered, forged, or fabricated (e.g., an attacker tampering with a message in transit, or fabricating fraudulent records).
- **Availability** — protects the accessibility of a system/data to authorized users when needed. It is directly violated by **Denial of Service (DoS)** attacks, which deliberately disrupt or block legitimate access to a resource.
- **Masquerade** also indirectly threatens integrity and availability, since an attacker posing as a legitimate entity may go on to falsify data or disrupt services once inside.

In short: falsification → breaks Integrity; masquerade → breaks Confidentiality (and can lead to Integrity/Availability breaches too); denial of service → breaks Availability. Together, these attack categories are simply concrete mechanisms by which the three CIA properties are undermined.

Q32. LCM of two integers (program)

```
#include <stdio.h>

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int lcm(int a, int b) {
    return (a / gcd(a, b)) * b;
}

int main() {
    int A, B;
    printf("Enter A and B: ");
    scanf("%d %d", &A, &B);
    printf("LCM(%d, %d) = %d\n", A, B, lcm(A, B));
    return 0;
}
```

Formula used: **LCM(A, B) = (A × B) / GCD(A, B)**.

Q33. SQL query — number of romantic comedy movies

Schema: Movies(mid, title, year), Genres(gid, genre), HasGenre(gid, mid).

Assuming “romantic comedy” is stored as a single genre value:

```
SELECT COUNT(DISTINCT hg.mid) AS romantic_comedy_count
FROM Genres g
JOIN HasGenre hg ON g.gid = hg.gid
WHERE g.genre = 'Romantic Comedy';
```

If “romantic” and “comedy” are two *separate* genres and a movie must have **both** to count as a romantic comedy:

```
SELECT COUNT(*) AS romantic_comedy_count
FROM (
    SELECT hg.mid
    FROM HasGenre hg JOIN Genres g ON hg.gid = g.gid
    WHERE g.genre IN ('Romantic', 'Comedy')
    GROUP BY hg.mid
    HAVING COUNT(DISTINCT g.genre) = 2
) AS rom_com_movies;
```

Q34. Java — overload, override, hide (Class A / Class B)

```
public class ClassA {
    public void m1() {}
    public void m2(int i) {}
    public void m3(int i) {}
    public static void m4(int i) {}
}

public class ClassB extends ClassA {
    public static void m1(int i) {} // (1)
    public void m2(int i) {} // (2)
    public void m3(String s) {} // (3)
    public static void m4(int i) {} // (4)
}
```

- **(1) m1(int i) in B vs m1() in A:** Different signature (different parameter list) — this is neither override nor overload of the *same method*, it is simply a **new/unrelated static method** in B that happens to share a name with an instance method in A (different signatures, and one is static while the other is instance — this does not compile as an override or valid overload relationship; it’s just a separate method definition — effectively **not related** to A’s m1()).
- **(2) m2(int i) in B vs m2(int i) in A:** Same signature, both instance methods → this **overrides** A’s m2.
- **(3) m3(String s) in B vs m3(int i) in A:** Same name, different parameter type → this **overloads** m3 (a new overloaded version is added in B; A’s int version is still inherited separately).
- **(4) m4(int i) in B vs m4(int i) in A:** Same signature, both **static** → static methods with the same signature in sub/superclass **hide** the superclass method (method hiding), rather than overriding it, since static methods are resolved at compile-time based on reference type, not runtime polymorphism.

Summary: m2 → override; m3 → overload; m4 → hide; m1 → unrelated/new method (mismatched signature and static/instance nature, not a valid override or overload of A’s m1()).

Q35. Adjacency matrix for complete binary tree on 7 vertices (heap-numbered)

For a complete binary tree with vertices numbered 1-7 (as in a binary heap), vertex i has children $2i$ and $2i+1$ (when ≤ 7): $1 \rightarrow 2, 3$ - $2 \rightarrow 4, 5$ - $3 \rightarrow 6, 7$

Adjacency Matrix (7×7, undirected: 1 = edge exists):

	1	2	3	4	5	6	7
1	0	1	1	0	0	0	0
2	1	0	0	1	1	0	0
3	1	0	0	0	0	1	1
4	0	1	0	0	0	0	0
5	0	1	0	0	0	0	0
6	0	0	1	0	0	0	0
7	0	0	1	0	0	0	0

Q36. Best OSI layer for end-to-end encryption

Recommendation: Application Layer (or, as a practical middle ground, the **Transport/Session layer using TLS**, sitting just above Transport).

Reasoning: - Development time & cost: Implementing encryption at the Application layer (e.g., using existing libraries like OpenSSL/TLS within the app, or protocols like HTTPS) is far quicker and cheaper than building custom encryption into lower layers (Network/Data Link), which would require modifying OS network stacks or hardware/firmware — a much larger undertaking. -

Software maintainability: Application-layer encryption keeps the security logic isolated within the application code, making it easier to update, patch, or swap cryptographic algorithms without touching the underlying OS or network infrastructure. Lower-layer changes (e.g., IPsec at the Network layer) require coordination across all network devices and are harder to maintain/update consistently. - **True end-to-end guarantee:** Only the Application layer (or TLS just below it) ensures data is encrypted **from the original sender's application all the way to the receiving application**, remaining encrypted through every intermediate hop/router. Lower-layer encryption (e.g., link-layer encryption) only protects data hop-by-hop, not truly end-to-end, since intermediate devices would need to decrypt/re-encrypt.

Conclusion: The Application layer (using well-established protocols like TLS/SSL) offers the best balance of true end-to-end protection, low development cost/time, and easier long-term maintainability.

Q37. Spoofing vs Sniffing

	Spoofing	Sniffing
Definition	Attacker fakes/forges identity (IP address, MAC address, email sender, etc.) to impersonate a trusted source	Attacker passively captures/monitors network traffic without altering it

Nature	Active attack (injects/sends fake data)	Passive attack (just listens)
Goal	Deceive the target into trusting false identity, enabling further attacks (MITM, session hijacking)	Steal sensitive information (passwords, data) by eavesdropping
Example	IP spoofing, email spoofing, ARP spoofing	Packet sniffing tools like Wireshark capturing unencrypted traffic on a network

Q38. SQL full form, aggregate functions, sample query & output

SQL = Structured Query Language.

Common aggregate functions: COUNT(), SUM(), AVG(), MIN(), MAX().

Sample table Students:

id	name	marks
1	Alice	80
2	Bob	70
3	Carol	90

Query:

```
SELECT AVG(marks) AS average_marks, MAX(marks) AS highest, COUNT(*)
AS total_students
FROM Students;
```

Output:

average_marks	highest	total_students
80.00	90	3

Q39. Linked List, Doubly Linked List, Circular Linked List

Singly Linked List: Each node has data + a pointer to the **next** node only.

[10|•]→[20|•]→[30|NULL]

Doubly Linked List: Each node has data, a pointer to the **next** node, and a pointer to the **previous** node — allows traversal in both directions.

NULL←[•|10|•]↔[•|20|•]↔[•|30|•]→NULL

Circular Linked List: The last node's next pointer points back to the **first** node (instead of NULL), forming a loop — can be singly or doubly circular.

[10|•]→[20|•]→[30|•]
 ↑_____|

Q40. Functionality of the OSI Model

The **OSI (Open Systems Interconnection)** model has 7 layers, each with a distinct role:

1. **Physical Layer:** Transmits raw bits over a physical medium (cables, radio signals); deals with voltage, signaling, connectors.
2. **Data Link Layer:** Provides node-to-node framing, error detection/correction, and MAC addressing (e.g., Ethernet, switches).
3. **Network Layer:** Handles logical addressing (IP) and routing of packets across different networks (e.g., routers, OSPF, IP).
4. **Transport Layer:** Provides end-to-end communication, reliability, flow control, and segmentation (e.g., TCP, UDP).
5. **Session Layer:** Establishes, manages, and terminates sessions/connections between applications.
6. **Presentation Layer:** Handles data translation, encryption/decryption, and compression (data format conversion between systems).
7. **Application Layer:** Provides network services directly to end-user applications (e.g., HTTP, FTP, DNS, email).

Overall, the model standardizes how different systems communicate by dividing the process into modular, well-defined layers.

Q41. Detecting a negative cycle in a weighted graph

Use the **Bellman-Ford algorithm**: 1. Initialize distance to the source as 0 and all other vertices as infinity. 2. Relax all edges (u, v, w) — i.e., update $\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w)$ — repeatedly for **$(V - 1)$** iterations, where V is the number of vertices (this guarantees shortest paths are found if no negative cycle exists). 3. Perform **one additional (V th) relaxation pass** over all edges. 4. **If any distance value can still be further reduced** in this extra pass (i.e., $\text{dist}[u] + w < \text{dist}[v]$ still holds for some edge), then the graph **contains a negative-weight cycle** reachable from the source — because in a graph without one, shortest paths would have already stabilized after $V - 1$ iterations.

(Alternative: the **Floyd-Warshall** algorithm can detect a negative cycle if, after computing all-pairs shortest paths, any diagonal element $\text{dist}[i][i] < 0$.)

Q42. 2nd largest number in C/C++

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter N: ");
    scanf("%d", &n);
    if (n < 2) {
        printf("Need at least 2 numbers.\n");
        return 0;
    }

    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    int first = -2147483648, second = -2147483648; // INT_MIN
    for (int i = 0; i < n; i++) {
        if (arr[i] > first) {
```

```

        second = first;
        first = arr[i];
    } else if (arr[i] > second && arr[i] != first) {
        second = arr[i];
    }
}
printf("2nd largest number = %d\n", second);
return 0;
}

```

Time complexity: $O(n)$, single pass, tracking the top two distinct values seen so far.

Q43. Count pairs (P, Q) from stream X such that $A \leq (P+Q) \leq B$, in $O(N \log N)$

Approach: 1. **Sort** the array X of N numbers — $O(N \log N)$. 2. Use the **two-pointer technique** to count pairs whose sum falls in $[A, B]$: - To count pairs with sum $\leq$ some limit L: use two pointers $i = 0$ (left) and $j = N-1$ (right). If $X[i] + X[j] \leq L$, then all elements between i and j paired with i also satisfy the condition (since array is sorted) $\rightarrow$ add $(j - i)$ pairs and increment i . Otherwise, decrement j . This runs in $O(N)$. - Let $\text{countLE}(L)$ = number of pairs with sum $\leq L$, computed via the two-pointer scan above. 3. **Answer** = $\text{countLE}(B) - \text{countLE}(A - 1)$ — the number of pairs with sum in the closed range $[A, B]$.

Pseudocode:

```

function countPairsLE(X, L):
    sort(X)
    i = 0, j = N-1, count = 0
    while i < j:
        if X[i] + X[j] <= L:
            count += (j - i)
            i += 1
        else:
            j -= 1
    return count

```

```

answer = countPairsLE(X, B) - countPairsLE(X, A - 1)

```

Total time complexity: $O(N \log N)$ for the sort + $O(N)$ for each two-pointer scan = $O(N \log N)$ overall.

Q44. ER Diagram — Library Management System

Entities & Attributes: - **Book** (Book_ID [PK], Title, Author, ISBN, Publisher, Copies_Available) - **Borrower** (Borrower_ID [PK], Name, Address, Phone, Email) - **Staff** (Staff_ID [PK], Name, Role) - **Borrowing** (Borrow_ID [PK], Book_ID [FK], Borrower_ID [FK], Borrow_Date, Due_Date, Return_Date)

Relationships: - Borrower **BORROWS** Book $\rightarrow$ Many-to-Many, resolved via the **Borrowing** junction entity. - Staff **MANAGES/ISSUES** Borrowing records $\rightarrow$ One-to-Many (one staff member processes many borrow transactions).

Text-based diagram:

```

[Book] ----< Borrowing >---- [Borrower]
      |
      ^
      |

```

[Staff]

Where ---< denotes the “many” side of the relationship, and Borrowing is the associative entity resolving the M:N relationship between Book and Borrower, additionally linked to Staff (who processed the transaction).

Q45. Print prime numbers from 1 to n

```
#include <stdio.h>

int isPrime(int num) {
    if (num < 2) return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return 0;
    }
    return 1;
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Prime numbers from 1 to %d:\n", n);
    for (int i = 2; i <= n; i++) {
        if (isPrime(i)) printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Q46. Floyd's Triangle (n = 5)

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```

Code:

```
#include <stdio.h>

int main() {
    int n = 5, num = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", num % 2);
            num++;
        }
        printf("\n");
    }
    return 0;
}
```

(Pattern: sequential numbers 1,2,3,... printed as number % 2, arranged in a triangle with row i having i values — reproducing the alternating 0/1 pattern shown.)

Q47. Sum of the series 1 + 2 + 4 + 7 + 11 + ... + N terms

The differences between consecutive terms are 1, 2, 3, 4, ... (i.e., $\text{term}(k) = \text{term}(k-1) + (k-1)$), so $\text{term}(k) = 1 + k(k-1)/2$.

```
#include <stdio.h>

int main() {
    int n, term = 1, sum = 0, diff = 0;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        sum += term;
        diff++;
        term += diff;
    }
    printf("Sum of series = %d\n", sum);
    return 0;
}
```

E.g., for $n = 5$: terms are 1, 2, 4, 7, 11 $\rightarrow$ sum = 25.

Q48. Polymorphism in OOP

Polymorphism ("many forms") allows objects of different classes to be treated through a common interface, with each responding to the same method call in its own way.

Types: 1. **Compile-time (Static) Polymorphism — Method**

Overloading: Multiple methods with the same name but different parameter lists within the same class; resolved at compile time. java
`void add(int a, int b) {} void add(double a, double b) {}` 2.

Runtime (Dynamic) Polymorphism — Method Overriding: A subclass provides its own implementation of a method already defined in its superclass; the actual method invoked is determined at runtime based on the object's actual type. java `class Animal { void sound() { System.out.println("Some sound"); } } class Dog extends Animal { void sound() { System.out.println("Bark"); } }` 3. **Operator**

Overloading (supported in languages like C++): the same operator behaves differently depending on operand types, e.g., + used for both integer addition and string concatenation, or overloaded for custom objects (e.g., adding two Complex number objects).

Q49. Multithreading

Multithreading is the ability of a program to execute multiple threads (smaller units of a process) concurrently, sharing the same process resources (memory space) while running independently.

Why it's used: - **Improved performance/responsiveness:** UI remains responsive while background tasks (e.g., file download, computation) run on separate threads. - **Better CPU utilization:** On multi-core processors, threads can run truly in parallel, speeding up execution of independent tasks. - **Efficient resource sharing:** Threads share memory/resources of the same process, making communication between them cheaper than between separate processes. - **Simplified modeling of concurrent tasks:** e.g., a web server handling multiple client requests simultaneously, each on its own thread.

Q50. Types of relations in DBMS

1. **One-to-One (1:1):** Each row in Table A relates to exactly one row

- in Table B, and vice versa. Example: Person ↔ Passport.
2. **One-to-Many (1:N)**: One row in Table A relates to many rows in Table B, but each row in B relates to only one row in A. Example: Department → Employees.
 3. **Many-to-One (N:1)**: The inverse of 1:N — many rows in A relate to a single row in B. Example: Many Students → one Class.
 4. **Many-to-Many (M:N)**: Many rows in A relate to many rows in B; implemented via a junction/bridge table. Example: Students ↔ Courses (via Enrollment).

Q51. Stack vs Queue + two problems each solves

	Stack	Queue
Order	LIFO (Last-In-First-Out)	FIFO (First-In-First-Out)
Operations	push, pop (both at same end — “top”)	enqueue (rear), dequeue (front)
Use case examples	Undo/Redo, expression evaluation	Task scheduling, print queue

Two problems solved using a Stack: 1. **Balanced parentheses checking** — push opening brackets, pop and match on closing brackets. 2. **Expression evaluation / conversion** (Infix to Postfix, or evaluating Postfix expressions).

Two problems solved using a Queue: 1. **BFS traversal of a graph/tree** — process nodes level by level. 2. **CPU/Process scheduling** (Round-Robin scheduling uses a queue of processes).

Q52. Recursion trace

```
#include <stdio.h>
void fun(int x) {
    if (x < 0) return;
    printf("%d\n", x--);
    fun(--x);
    printf("%d\n", x);
}
int main() {
    fun(5);
    return 0;
}
```

Trace: - fun(5): print x -- → prints **5**, x becomes 4. Call fun(--x) → fun(3). - fun(3): print **3**, x becomes 2. Call fun(--x) → fun(1). - fun(1): print **1**, x becomes 0. Call fun(--x) → fun(-1). - fun(-1): x < 0 → return immediately (nothing printed). - back in fun(1) (x is now 0 after the call returns, unaffected by the callee — x was passed by value): print x → **0**. - back in fun(3) (x = 2): print x → **2**. - back in fun(5) (x = 4): print x → **4**.

Full Output:

5
3
1
0
2
4

Q53. Fibonacci series using Dynamic Programming

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);

    int fib[n + 1];
    fib[0] = 0;
    if (n >= 1) fib[1] = 1;

    for (int i = 2; i <= n; i++) {
        fib[i] = fib[i - 1] + fib[i - 2]; // reuse previously
        computed values (memoization/tabulation)
    }

    printf("Fibonacci series: ");
    for (int i = 0; i <= n; i++) printf("%d ", fib[i]);
    printf("\n");

    return 0;
}
```

Why DP: Naive recursion recomputes the same subproblems exponentially ($O(2^n)$). Storing each `fib[i]` once (bottom-up tabulation) reduces this to **$O(n)$ time, $O(n)$ space** (or $O(1)$ space using just two variables instead of an array).

Q54. Hash Function

A **Hash function** is a function that takes an input (key) of arbitrary size and maps it to a fixed-size output value (the hash code/hash value), typically used to determine the index/bucket in a **hash table** where the corresponding data should be stored or looked up.

Key properties:

- **Deterministic:** the same input always produces the same hash output.
- **Fast to compute.**
- **Uniform distribution:** spreads keys evenly across the hash table to minimize collisions.
- **Minimizes collisions:** ideally, different keys map to different hash values (though collisions are unavoidable in general and are handled via chaining or open addressing/probing).

Example: $h(k) = k \bmod \text{tableSize}$ maps any integer key k to an index between 0 and `tableSize - 1`.

Hash functions are the backbone of hash tables (used for fast $O(1)$ average-case lookup/insert/delete), and are also used in cryptography (e.g., SHA-256), data integrity checks, and password storage.